



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Lívia Corazza Ferrão - 21202119

INE5429 - Segurança da computação

Pipeline de CI/CD seguro

Florianópolis
2026

1. Descrição do Sistema e Ferramental

1.1 Arquitetura do Sistema

O sistema escolhido é uma aplicação web de gerenciamento de estágios. A aplicação é um projeto pessoal que possui vulnerabilidades e que conseguiu demonstrar a eficácia de ferramentas de análise de segurança em um pipeline de CI/CD.

A arquitetura é composta por três camadas containerizadas com Docker:

- Frontend: Aplicação React servida via Nginx
- Backend: API RESTful em Node.js com Express
- Banco de dados: MongoDB 6.0

1.2 Stack Tecnológico

O desenvolvimento do ecossistema da aplicação utiliza as seguintes tecnologias distribuídas por suas respectivas camadas e funções:

- **Frontend:** Desenvolvido em React combinado com Vite, utilizando a versão 18+.
- **Backend:** Construído em Node.js com o framework Express, rodando na versão 18 LTS.
- **Banco de Dados:** Utiliza o MongoDB na versão 6.0.
- **Servidor Web:** Nginx na versão 1.24.
- **Autenticação:** Gerenciada via tokens JWT (biblioteca jsonwebtoken) na versão 9.0.2.
- **Containerização:** Estruturada com Docker e Docker Compose.
- **CI/CD:** Automatizado através do GitHub Actions.

1.3 Plataforma de CI/CD e Ferramentas

A pipeline foi implementada no GitHub Actions com 6 jobs executados em sequência lógica. Inicialmente, quatro varreduras (scans) de segurança rodam em paralelo, seguidas pelo job de build e, por último, pela análise dinâmica (DAST). Os detalhes de cada etapa estão descritos a seguir:

- **Secret Detection:** Executado pela ferramenta **Gitleaks v2**, realiza uma análise focada na detecção de segredos e credenciais que possam estar expostos no código-fonte.
- **SCA (Software Composition Analysis):** Utiliza a ferramenta **Trivy** para fazer a análise de composição de software, identificando vulnerabilidades nas dependências e pacotes utilizados.
- **SAST (Static Application Security Testing):** Emprega o **Semgrep** para efetuar a análise estática de segurança diretamente no código-fonte desenvolvido.
- **IaC Scan (Infrastructure as Code):** Roda a ferramenta **Checkov** voltada para a análise de segurança da infraestrutura como código, avaliando as configurações dos Dockerfiles.

- **Build:** Utiliza o ecossistema do **Node.js 18 e npm** para realizar a compilação e validação do código do frontend.
- **DAST (Dynamic Application Security Testing):** Conclui o processo com o **OWASP ZAP**, que executa uma análise dinâmica de segurança simulando ataques na aplicação enquanto ela está em execução.

Repositório: <https://github.com/Livia-ferrao/INE5429-Seguranca>

2. Evidências de execução

A pipeline foi executada com sucesso no GitHub Actions. Todos os 6 jobs foram disparados automaticamente via push na branch main. A seguir são apresentados os resultados consolidados de cada ferramenta.

2.1 Secret Detection - Gitleaks

O Gitleaks foi executado com varredura completa do histórico de commits analisando todas as branches e commits do repositório. A ferramenta detectou a presença de uma credencial hardcoded classificada como genetic-api-key, com ocorrências em 4 arquivos distintos. O segredo detectado é a chave `ghp_R9kT2mXvL4nQ8pYwZ6jB3cF5hA1dE7sG0uI`, demonstrando um vazamento de credencial.

Secret Detection (Gitleaks) summary							
 Gitleaks detected secrets 							
Rule ID	Commit	Secret URL	Start Line	Author	Date	Email	File
generic-api-key	c7a02c1	View Secret	6	Livia-ferrao	2026-06-24T19:04:33Z	livia.ferrao16@hotmail.com	.env.example
generic-api-key	c7a02c1	View Secret	29	Livia-ferrao	2026-06-24T19:04:33Z	livia.ferrao16@hotmail.com	docker-compose.yml
generic-api-key	c7a02c1	View Secret	4	Livia-ferrao	2026-06-24T19:04:33Z	livia.ferrao16@hotmail.com	app.js
generic-api-key	c7a02c1	View Secret	29	Livia-ferrao	2026-06-24T19:04:33Z	livia.ferrao16@hotmail.com	docker-compose.ci.yml

[Job summary generated at run-time](#)

2.2 SCA - Trivy

O Trivy foi utilizado para realizar a análise de composição de software (SCA), verificando as dependências presentes nos arquivos package-lock.json do backend e frontend da aplicação. A ferramenta identificou vulnerabilidades conhecidas (CVEs) em bibliotecas utilizadas pelo projeto.

No total, foram encontradas 63 vulnerabilidades, sendo 42 no frontend e 21 no backend. A análise foi integrada ao pipeline CI/CD, configurado para retornar exit code 1 quando vulnerabilidades são encontradas. Entretanto, o workflow utiliza `continue-on-error: true`, permitindo que a execução continue para as próximas etapas mesmo com falhas identificadas.

Report Summary			
Target	Type	Vulnerabilities	Secrets
frontend/package-lock.json	npm	42	-
package-lock.json	npm	21	-

No backend, as principais vulnerabilidades encontradas foram:

- **mongoose@7.5.4 – CVE-2025-23061 (CRITICAL)**: vulnerabilidade de *Search Injection*, podendo permitir manipulação indevida de consultas.
- **lodash@4.17.20 – CVE-2021-23337 (HIGH)**: possibilidade de execução de comandos através de templates.
- **lodash@4.17.20 – CVE-2026-4800 (HIGH)**: execução arbitrária de código por meio de manipulação de imports em templates.
- **jws@3.2.2 – CVE-2025-65945 (HIGH)**: falha na validação de assinaturas HS256, podendo comprometer a verificação de tokens.
- **path-to-regexp@0.1.7 – CVE-2024-45296 (HIGH)**: vulnerabilidade de ReDoS (*Regular Expression Denial of Service*).
- **body-parser@1.20.1 – CVE-2024-45590 (HIGH)**: possibilidade de negação de serviço.

No frontend, foram identificadas vulnerabilidades principalmente em bibliotecas responsáveis por comunicação HTTP e roteamento:

- **form-data@4.0.0 – CVE-2025-7783 (CRITICAL)**: uso de função insegura na geração de valores aleatórios.
- **axios@1.6.2 – CVE-2024-39338 (HIGH)**: vulnerabilidade de SSRF (*Server-Side Request Forgery*).
- **axios@1.6.2 – CVE-2026-42033 (HIGH)**: possibilidade de manipulação do transporte HTTP através de *Prototype Pollution*.
- **@remix-run/router@1.11.0 – CVE-2026-22029 (HIGH)**: vulnerabilidade de XSS associada a redirecionamentos inseguros.
- **lodash@4.17.21 – CVE-2026-4800 (HIGH)**: execução arbitrária de código através de templates.

A análise demonstrou a importância da utilização de SCA no pipeline, pois permite identificar dependências vulneráveis antes da implantação da aplicação. As falhas encontradas devem ser tratadas com atualização das bibliotecas afetadas, aplicação de correções de segurança e substituição de versões descontinuadas quando necessário.

2.3 SAST - Semgrep

O Semgrep analisou 85 arquivos rastreados pelo git utilizando 580 regras dos rulesets p/nodejs, p/owasp-top-ten e p/secrets.

Debugging Info

SCAN ENVIRONMENT

versions - semgrep 1.36.0 on python 3.11.4

environment - running in environment github-actions, triggering event is push

Scan Status

Scanning 85 files tracked by git with 580 Code rules:

Language	Rules	Files	Origin	Rules
<multilang>	48	170	Community	580
js	66	51		
json	3	4		
yaml	19	3		
dockerfile	4	2		
html	1	1		

A varredura identificou 1 finding bloqueante:

1 Blocking Code Finding

```
nginx.conf
generic.nginx.security.request-host-used.request-host-used
  '$http_host' and '$host' variables may contain a malicious value from
attacker controlled
  'Host' request header. Use an explicitly configured host value or a
allow list for
validation.
Details: https://sg.run/4x3Z

10| proxy_set_header Host $host;
```

BLOCKING CODE RULES FIRED:

generic.nginx.security.request-host-used.request-host-used

A regra alerta que as variáveis \$host e \$http_host podem receber valores manipulados pelo cliente através do cabeçalho HTTP Host. Em determinados

cenários isso pode possibilitar ataques de Host Header Injection, especialmente quando a aplicação utiliza esse valor para geração de URLs, redirecionamentos ou validações de domínio. Embora o finding tenha sido classificado como bloqueante pela ferramenta, trata-se de uma configuração comum em servidores Nginx atuando como proxy reverso. O impacto real depende do contexto da aplicação e da existência de validações adicionais.

A aplicação contém uma vulnerabilidade de NoSQL Injection no endpoint `/api/v1/estagios/busca`. Nesse trecho, todos os parâmetros recebidos através de `req.query` são armazenados na variável `filtro` e utilizados diretamente na consulta ao banco de dados através do método `Estagio.find(filtro)`. Como não existe validação dos campos recebidos, um atacante pode enviar operadores especiais do MongoDB, como `$ne`, `$gt`, `$regex` e outros, alterando a lógica da consulta originalmente prevista pela aplicação. Isso pode permitir o retorno indevido de registros, a manipulação dos critérios de busca ou a realização de consultas não autorizadas.

A expectativa era que o Semgrep identificasse esse padrão inseguro durante a análise SAST, uma vez que a consulta utiliza diretamente dados controlados pelo usuário em uma operação de banco de dados. Entretanto, a execução da ferramenta não gerou alertas relacionados a NoSQL Injection, detectando apenas uma configuração potencialmente insegura no Nginx referente ao uso dos cabeçalhos `$host` e `$http_host`. Esse resultado evidencia uma limitação das análises estáticas baseadas em regras.

2.4 IaC Scan - Checkov

O Checkov foi executado para análise de Infraestrutura como Código (IaC), inspecionando os Dockerfiles do backend e do frontend. A ferramenta realizou 67 verificações de segurança, das quais 63 foram aprovadas e 4 apresentaram falhas.

As falhas identificadas foram:

- **frontend/Dockerfile - CKV_DOCKER_2:** ausência da instrução `HEALTHCHECK`, responsável por monitorar a saúde do container.
- **frontend/Dockerfile - CKV_DOCKER_3:** execução do container sem a definição de um usuário não privilegiado, utilizando o usuário padrão da imagem.
- **Dockerfile (backend) - CKV_DOCKER_2:** ausência da instrução `HEALTHCHECK`.
- **Dockerfile (backend) - CKV_DOCKER_3:** execução do container utilizando privilégios elevados (usuário `root`).

Essas falhas representam más práticas de segurança em ambientes containerizados. A ausência de um mecanismo de verificação de saúde dificulta a detecção automática de falhas da aplicação, enquanto a execução como usuário `root` aumenta o impacto potencial de uma exploração bem-sucedida do container.

Apesar das não conformidades encontradas, o pipeline não foi interrompido, pois a execução do Checkov foi configurada com `soft_fail: true`, permitindo o registro dos resultados sem bloquear os demais estágios da pipeline.

```

Check: CKV_DOCKER_3: "Ensure that a user for the container has been created"
FAILED for resource: /frontend/Dockerfile.
Error: File: /frontend/Dockerfile:1-17
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-a-user-for-the-container-has-been-created

1 | FROM node:18-alpine AS build
2 |
3 | WORKDIR /app
4 |
5 | COPY package*.json ./
6 | RUN npm install
7 |
8 | COPY . .
9 | RUN npm run build
10 |
11 | FROM nginx:1.24-alpine
12 |
13 | COPY --from=build /app/dist /usr/share/nginx/html
14 |
15 | EXPOSE 80
16 |
17 | CMD ["nginx", "-g", "daemon off;"]

Check: CKV_DOCKER_2: "Ensure that HEALTHCHECK instructions have been added to container images"
FAILED for resource: /frontend/Dockerfile.
Error: File: /frontend/Dockerfile:1-17
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-healthcheck-instructions-have-been-added-to-container-images

1 | FROM node:18-alpine AS build
2 |
3 | WORKDIR /app
4 |
5 | COPY package*.json ./
6 | RUN npm install
7 |
8 | COPY . .
9 | RUN npm run build
10 |
11 | FROM nginx:1.24-alpine
12 |
13 | COPY --from=build /app/dist /usr/share/nginx/html
14 |
15 | EXPOSE 80
16 |
17 | CMD ["nginx", "-g", "daemon off;"]

```

```

Error: File: /Dockerfile:1-13
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-a-user-for-the-container-has-been-created

1 | FROM node:18-alpine
2 |
3 | WORKDIR /app
4 |
5 | COPY package*.json ./
6 |
7 | RUN npm install
8 |
9 | COPY . .
10 |
11 | EXPOSE 4000
12 |
13 | CMD ["node", "app.js"]

Check: CKV_DOCKER_2: "Ensure that HEALTHCHECK instructions have been added to container images"
FAILED for resource: /Dockerfile.
Error: File: /Dockerfile:1-13
Guide: https://docs.prismacloud.io/en/enterprise-edition/policy-reference/docker-policies/docker-policy-index/ensure-that-healthcheck-instructions-have-been-added-to-container-images

1 | FROM node:18-alpine
2 |
3 | WORKDIR /app
4 |
5 | COPY package*.json ./
6 |
7 | RUN npm install
8 |
9 | COPY . .
10 |
11 | EXPOSE 4000
12 |
13 | CMD ["node", "app.js"]

```

2.5 DAST - OWASP ZAP

A análise dinâmica de segurança (DAST) foi realizada utilizando a ferramenta **OWASP ZAP** em um ambiente de execução da aplicação. O processo incluiu a inicialização dos contêineres Docker do sistema e a execução de um escaneamento passivo sobre a aplicação disponível em <http://localhost:4000>.

O resultado indicou que não foram encontradas vulnerabilidades críticas ou de alta severidade exploráveis durante a análise dinâmica. Das 70 verificações realizadas, 61 foram aprovadas sem a identificação de problemas. Entretanto, foram detectados 9 alertas classificados como advertências (*warnings*), relacionados principalmente à

ausência de cabeçalhos HTTP de segurança e possíveis exposições de informações do servidor.

Falhas identificadas

- Ausência do cabeçalho X-Frame-Options ou mecanismo equivalente de proteção contra ataques de *Clickjacking*.
- Ausência do cabeçalho X-Content-Type-Options, que auxilia na prevenção de ataques de *MIME Sniffing*.
- Exposição de informações do servidor por meio do cabeçalho X-Powered-By.
- Ausência de uma política de segurança de conteúdo (Content Security Policy – CSP).
- Conteúdo passível de armazenamento em cache sem diretivas explícitas de controle.
- Ausência do cabeçalho Permissions-Policy, utilizado para restringir recursos do navegador.
- Possível configuração inadequada de compartilhamento entre domínios (*Cross-Domain Misconfiguration*).
- Ausência ou configuração inválida do cabeçalho Cross-Origin-Embedder-Policy (COEP).
- Ausência do cabeçalho Sec-Fetch-Dest, utilizado em mecanismos modernos de proteção de requisições.

Além das vulnerabilidades identificadas, observou-se uma falha no pipeline relacionada ao upload do artefato do relatório do ZAP para o GitHub Actions, causada por um nome de artefato considerado inválido pela plataforma. Contudo, esse problema não comprometeu a execução da análise de segurança nem os resultados obtidos.


```

WARN-NEW: Missing Anti-clickjacking Header [10020] x 1
  http://localhost:4000 (200 OK)
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 1
  http://localhost:4000 (200 OK)
WARN-NEW: Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037] x 4
  http://localhost:4000 (200 OK)
  http://localhost:4000/api-docs (404 Not Found)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 4
  http://localhost:4000 (200 OK)
  http://localhost:4000/api-docs (404 Not Found)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
WARN-NEW: Storable and Cacheable Content [10049] x 4
  http://localhost:4000 (200 OK)
  http://localhost:4000/api-docs (404 Not Found)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
WARN-NEW: Permissions Policy Header Not Set [10063] x 4
  http://localhost:4000 (200 OK)
  http://localhost:4000/api-docs (404 Not Found)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
WARN-NEW: Cross-Domain Misconfiguration [10098] x 4
  http://localhost:4000 (200 OK)
  http://localhost:4000/api-docs (404 Not Found)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
WARN-NEW: Cross-Origin-Embedder-Policy Header Missing or Invalid [90004] x 2
  http://localhost:4000 (200 OK)
  http://localhost:4000 (200 OK)
WARN-NEW: Sec-Fetch-Dest Header is Missing [90005] x 12
  http://localhost:4000 (200 OK)
  http://localhost:4000/robots.txt (404 Not Found)
  http://localhost:4000/sitemap.xml (404 Not Found)
  http://localhost:4000 (200 OK)
  http://localhost:4000/robots.txt (404 Not Found)
FAIL-NEW: 0      FAIL-INPROG: 0  WARN-NEW: 9      WARN-INPROG: 0  INFO: 0  IGNORE: 0      PASS: 61

```

3. Análise de Falsos Positivos e Alertas Irrelevantes

Após a execução das ferramentas de segurança, foi realizada uma análise manual dos alertas gerados para diferenciar vulnerabilidades reais de falsos positivos ou achados com baixo impacto no contexto da aplicação.

DAST: Utilizando OWASP ZAP, grande parte dos alertas identificados está relacionada à ausência de cabeçalhos HTTP de segurança. Embora esses alertas sejam boas práticas recomendadas, eles não representam necessariamente vulnerabilidades exploráveis na aplicação atual.

O alerta referente à ausência do cabeçalho X-Frame-Options foi considerado de baixo impacto, pois a aplicação não possui funcionalidades críticas que dependam de incorporação em páginas externas. A inclusão do cabeçalho pode ser aplicada

como melhoria de segurança, mas sua ausência não permitiu explorar uma falha funcional durante o teste.

A ausência do cabeçalho X-Content-Type-Options também foi classificada como um alerta de configuração. O problema pode aumentar riscos relacionados à interpretação incorreta de tipos de conteúdo pelo navegador, porém não foi identificada exploração direta na aplicação.

O alerta relacionado ao cabeçalho X-Powered-By representa uma exposição de informação sobre a tecnologia utilizada pelo servidor. Apesar de ser uma prática recomendada remover esse cabeçalho para reduzir informações disponíveis a um atacante, a informação exposta não representa uma vulnerabilidade crítica isoladamente.

Os alertas referentes à ausência de Content Security Policy (CSP), Permissions-Policy, Cross-Origin-Embedder-Policy (COEP) e Sec-Fetch-Dest foram considerados melhorias de endurecimento (*hardening*) da aplicação. Esses mecanismos aumentam a proteção do navegador contra determinados tipos de ataques, mas a ausência deles não indica, por si só, uma falha explorável identificada durante o teste.

SAST: Utilizando Semgrep, o alerta relacionado ao uso das variáveis \$host e \$http_host no Nginx foi avaliado como um possível falso positivo. A ferramenta identificou o risco de Host Header Injection, porém o uso dessas variáveis ocorre em um cenário comum de proxy reverso. Como a aplicação não utiliza o valor recebido do cabeçalho Host para geração dinâmica de links, autenticação ou redirecionamentos sensíveis, o risco de exploração foi considerado reduzido.

SCA: O Trivy identificou vulnerabilidades em dependências utilizadas pelo backend e frontend da aplicação. Após a análise dos alertas, foi realizada uma verificação para identificar quais bibliotecas possuem utilização direta no código-fonte e quais estão presentes apenas como dependências transitivas.

Algumas vulnerabilidades foram encontradas em bibliotecas que não possuem uso direto identificado no código-fonte da aplicação. Por exemplo, embora o lodash esteja declarado como dependência do projeto e apresente vulnerabilidades relacionadas à manipulação de templates, não foram identificados imports ou chamadas às funcionalidades vulneráveis durante a análise do código. Dessa forma, o risco de exploração foi considerado reduzido no contexto atual da aplicação.

IAC: Os alertas relacionados à ausência de HEALTHCHECK nos Dockerfiles não representam uma vulnerabilidade explorável diretamente, mas uma deficiência operacional. A ausência dessa instrução dificulta a detecção automática de containers em estado inadequado, porém não aumenta diretamente a superfície de ataque.

4. Identificação e Correção de Falhas Reais

Após a análise dos resultados obtidos pelas ferramentas de segurança, foram selecionadas as falhas consideradas reais e com impacto relevante para a aplicação. A priorização levou em consideração a severidade do alerta, a

possibilidade de exploração e o contexto de uso da vulnerabilidade. Alterações foram realizadas em ambiente local e as imagens demonstram os resultados obtidos.

4.1 Exposição de Credencial - Gitleaks

A análise realizada pelo Gitleaks identificou uma credencial armazenada diretamente no código-fonte do projeto. Esse tipo de falha representa um risco elevado, pois informações sensíveis podem ser utilizadas por terceiros caso o repositório seja acessado.

A correção consiste na remoção da credencial do código e substituição por uma variável de ambiente. Dessa forma, o valor sensível deixa de ser versionado junto ao código-fonte.

Em `app.js` remoção da variável pois não está sendo usada.

```
const JWT_SECRET_FALLBACK = "ghp_R9kT2mXvL4nQ8pYwZ6jB3cF5hA1dE7sG0uI";
```

Nos Dockers Compose substituído por JWT SECRET.

```
environment:
  - PORT=4000
  - MONGO_URI=mongodb://root:rootpassword@localhost:27017/estagios_ine?authSource=admin
  - JWT_SECRET=${JWT_SECRET}
  - JWT_EXPIRES_IN=30d
```

No `env.example`, chave real deve ser substituída por string indicando o que deve-se fazer ao criar o `.env`

```
JWT_SECRET=coloque_sua_chave_aqui
```

4.2 Vulnerabilidades em Dependências - Trivy

O Trivy identificou diversas vulnerabilidades em dependências utilizadas pelo backend e frontend. A principal ação de correção consiste na atualização das bibliotecas para versões mais recentes que contenham os patches de segurança disponibilizados pelos mantenedores.

As dependências consideradas prioritárias para atualização foram:

- **mongoose:** atualização para uma versão corrigida para remover riscos relacionados a Search Injection.
- **axios:** atualização para versões corrigidas contra vulnerabilidades relacionadas a SSRF e manipulação de requisições HTTP.

4.3 Execução de Containers com Privilégios Elevados - Checkov

O Checkov identificou a ausência de mecanismos de monitoramento de saúde dos containers (HEALTHCHECK) e a execução das aplicações com privilégios excessivos. Essas configurações representam más práticas de segurança, pois

dificultam a detecção automática de falhas e aumentam o impacto potencial de uma invasão.

A correção consiste na inclusão de instruções HEALTHCHECK nos Dockerfiles do backend e frontend, permitindo que o orquestrador monitore continuamente a disponibilidade dos serviços. Também necessita-se criar usuários não privilegiados para execução das aplicações, eliminando a utilização do usuário root dentro dos containers.

Novo Dockerfile frontend:

```
FROM node:18-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:1.24-alpine

COPY --from=build /app/dist /usr/share/nginx/html

RUN addgroup -S nginxgroup && adduser -S nginxuser -G nginxgroup

RUN chown -R nginxuser:nginxgroup /usr/share/nginx/html \
    && chown -R nginxuser:nginxgroup /var/cache/nginx \
    && chown -R nginxuser:nginxgroup /var/log/nginx \
    && touch /var/run/nginx.pid \
    && chown nginxuser:nginxgroup /var/run/nginx.pid

USER nginxuser

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost || exit 1

CMD ["nginx", "-g", "daemon off;"]
```

Novo Dockerfile backend:

```

FROM node:18-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

RUN chown -R appuser:appgroup /app

USER appuser

EXPOSE 4000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:4000 || exit 1

CMD ["node", "app.js"]

```

4.4 Implementação de Cabeçalhos HTTP de Segurança - OWASP ZAP

Embora os alertas encontrados pelo ZAP tenham sido classificados principalmente como recomendações, algumas melhorias foram aplicadas para aumentar a proteção da aplicação.

Foram adicionados ou recomendados os seguintes cabeçalhos:

- X-Frame-Options para reduzir riscos de Clickjacking;
- X-Content-Type-Options para evitar interpretação incorreta de conteúdo;
- Content-Security-Policy para restringir fontes de execução no navegador;
- Remoção do cabeçalho X-Powered-By para evitar exposição da tecnologia utilizada.

A implementação foi realizada por meio da adição do middleware Helmet na aplicação:

```

app.use(express.json());

app.use(helmet());
app.use(cors({ origin: "*" }));
app.use(xss());

```

Também foi removido o cabeçalho X-Powered-By, que expunha informações sobre a tecnologia utilizada no backend, reduzindo a quantidade de informações disponíveis para um potencial atacante.

```
// Express

const express = require("express");
const app = express();

app.disable("x-powered-by");
```

Essas alterações não corrigem vulnerabilidades críticas diretamente exploráveis, mas fortalecem a configuração de segurança da aplicação, reduzem a superfície de ataque e seguem as recomendações do OWASP para hardening de aplicações web.